I'll create a practical PRD for an AI Eval product, assuming the target users are AI Product Managers, AI engineers, QA, and domain reviewers. I'll also benchmark current eval concepts/tools so the PRD reflects today's market and not just generic product thinking.

Thought for 22s ›

Below is a practical PRD for an **AI Eval product** you can use as a starting point. I framed it as a product that helps **Product Managers, AI engineers, QA, and domain experts** define, run, review, and monitor AI evaluations.

Edit

Product Requirements Document: AI EvalOps Platform

1. Product Name

EvalPilot
An AI evaluation platform that helps teams define what "good" looks like, test AI outputs, measure quality, detect hallucinations, and continuously improve AI products before and after launch.

---

2. Product Summary

AI products are harder to test than traditional software because outputs can vary for the same input. Traditional pass/fail testing is not enough for LLM-based applications, chatbots, copilots, RAG systems, and AI agents. OpenAI's eval guidance describes evals as structured tests for measuring model performance despite AI variability, and recommends an evaluation loop: define objectives, collect datasets, define metrics, run evals, compare results, and iterate.

EvalPilot will provide a structured, PM-friendly platform where teams can create golden datasets, generate synthetic test cases, define evaluation rubrics, run automated and human evaluations, compare model/prompt versions, and monitor production quality over time.

---

3. Problem Statement

AI teams are shipping LLM-powered features without a consistent way to measure output quality. Many teams still rely on manual testing, demo-based confidence, or subjective reviews. This leads to production issues such as hallucinations, irrelevant answers, unsafe responses, incorrect tool usage, poor tone, and inconsistent user experience.

Existing engineering-focused tools support evals, tracing, and observability, but many Product Managers and business stakeholders struggle to define quality criteria, review outputs, and connect eval results to product decisions.

---

4. Opportunity

There is a growing need for a bridge between product thinking and AI engineering execution.

Current eval platforms and frameworks commonly support concepts like datasets, evaluators, experiments, human review, code-based rules, LLM-as-judge scoring, and production monitoring. LangSmith's evaluation workflow, for example, includes creating datasets from curated cases, historical traces, or synthetic data; defining evaluators such as human review, code rules, LLM-as-judge, and pairwise comparison; running experiments; and analyzing results. Phoenix also supports deterministic evaluators and LLM-as-judge evaluators against traces, experiment results, and datasets.

The opportunity for EvalPilot is to make this workflow accessible to cross-functional AI product teams, especially PMs who need to define success criteria and make go/no-go launch decisions.

---

5. Target Users

Primary Users

AI Product Manager
Defines product goals, quality criteria, customer experience

expectations, and launch readiness.

AI Engineer / ML Engineer
Connects model outputs, prompts, traces, APIs, and experiment runs into the eval platform.

QA Analyst / Test Engineer
Creates test cases, reviews outputs, validates regressions, and tracks defects.

Domain Expert / Business Reviewer
Reviews outputs for accuracy, compliance, policy, tone, and business correctness.

Secondary Users

Engineering Manager
Reviews quality trends and release readiness.

Compliance / Risk Reviewer
Reviews safety, policy, privacy, and regulated-domain outputs.

Customer Support Lead
Validates support-bot responses and escalations.

---

6. Goals

1. Help AI teams define quality criteria for LLM applications.
2. Create and manage golden test datasets.
3. Generate synthetic test scenarios to expand coverage.
4. Run automated evals using deterministic checks and LLM-as-judge.
5. Support human review workflows for subjective or high-risk outputs.
6. Detect hallucinations, irrelevant responses, unsafe outputs, and tone issues.
7. Compare prompt, model, retrieval, and workflow versions.
8. Provide dashboards for launch readiness and production monitoring.
9. Convert failed evals into improvement tasks for product and engineering teams.

7. Non-Goals for MVP

1. The product will not train or fine-tune models directly in MVP.
2. The product will not replace existing observability platforms in MVP.
3. The product will not become a full annotation platform in MVP.
4. The product will not support every AI modality in MVP; initial scope is text-based LLM applications.
5. The product will not provide legal/compliance certification; it will provide evidence and workflow support.

---

8. MVP Scope

The MVP should focus on a simple but complete evaluation lifecycle:

Create dataset → Define rubric → Run eval → Review results → Compare versions → Track improvements

MVP Modules

1. Project Setup
2. Golden Dataset Manager
3. Synthetic Test Case Generator
4. Evaluation Rubric Builder
5. Eval Runner
6. Human Review Queue
7. Result Dashboard
8. Regression Comparison
9. Issue Tracking / Export
10. Basic Production Sampling

---

9. Key Use Cases

Use Case 1: PM Defines AI Quality Criteria

A Product Manager creates an eval project for a customer support chatbot. The PM defines key dimensions such as correctness,

helpfulness, tone, policy compliance, hallucination risk, and escalation behavior.

Use Case 2: Team Creates Golden Dataset

The team uploads real examples or manually creates test cases. Each test case includes user input, expected behavior, ideal answer, business policy, tags, and severity.

Use Case 3: Synthetic Data Expansion

The product generates variations of existing golden examples. For example, one refund request can become multiple emotional, incomplete, multilingual, or edge-case prompts.

Use Case 4: Automated Evaluation

The team runs evals against a model, prompt, RAG pipeline, or agent. The system scores outputs using rules, rubric-based LLM judges, and optional reference answers.

Use Case 5: Human Review

Low-confidence, high-risk, or failed outputs are routed to human reviewers. Reviewers score outputs and add comments.

Use Case 6: Prompt/Model Regression Testing

Before launching a new prompt or model version, the team compares eval results against the previous production version.

Use Case 7: Production Monitoring

The product samples live AI interactions, scores them, flags potential issues, and adds failed cases back into the golden dataset.

---

10. Core User Stories

Project Setup

ID	User Story	Priority

US-01	As a PM, I want to create an eval project for an AI feature so that my team can track quality in one place.	P0
US-02	As a PM, I want to define the AI use case type such as chatbot, RAG, summarization, content generation, or agent so that the platform can recommend relevant metrics.	P0
US-03	As an admin, I want to invite engineers, reviewers, and stakeholders so that each role can contribute to the eval workflow.	P1

Golden Dataset

ID	User Story	Priority
US-04	As a PM, I want to create golden examples with input, expected output, tags, and severity so that the team has a trusted evaluation baseline.	P0
US-05	As a QA analyst, I want to bulk upload test cases using CSV or JSON so that I can quickly create datasets.	P0
US-06	As a reviewer, I want to tag cases as edge case, policy risk, hallucination risk, tone issue, or escalation case so that failures are easier to analyze.	P1

Synthetic Test Generation

ID	User Story	Priority
US-07	As a PM, I want to generate variations of a golden test case so that I can increase test coverage.	P0
US-08	As a PM, I want to control synthetic data generation by tone, language, user emotion, complexity, and edge-case type so that generated tests match real-world scenarios.	P1

US-09	As a reviewer, I want to approve or reject synthetic test cases before they become part of the official eval dataset.	P0

Rubric Builder

ID	User Story	Priority
US-10	As a PM, I want to create evaluation rubrics for accuracy, helpfulness, safety, tone, groundedness, and policy compliance so that quality is clearly defined.	P0
US-11	As a PM, I want to define scoring rules from 1–5 or pass/fail so that eval results are consistent.	P0
US-12	As a compliance reviewer, I want to mark some criteria as launch blockers so that critical failures cannot be ignored.	P0

Eval Runner

ID	User Story	Priority
US-13	As an engineer, I want to connect the eval platform to an API endpoint, prompt, or model configuration so that outputs can be tested automatically.	P0
US-14	As an engineer, I want to run deterministic checks such as exact match, regex, JSON validity, forbidden words, and policy keyword checks so that objective failures are caught.	P0
US-15	As a PM, I want to run LLM-as-judge evaluations against a rubric so that subjective quality dimensions can be scored at scale.	P0
US-16	As an engineer, I want to run the same dataset against two versions so that I can compare model, prompt, or retrieval changes.	P0

Human Review

ID	User Story	Priority
US-17	As a reviewer, I want a queue of failed or low-confidence outputs so that I can manually grade them.	P0
US-18	As a reviewer, I want to add comments and corrected answers so that the team can improve prompts or policies.	P0
US-19	As a PM, I want human review scores to be compared with AI judge scores so that we can calibrate the judge.	P1

Results Dashboard

ID	User Story	Priority
US-20	As a PM, I want a dashboard showing pass rate, hallucination rate, average quality score, safety failure rate, and regression trend so that I can make launch decisions.	P0
US-21	As an engineering manager, I want to compare eval results across versions so that I can understand whether the new version improved or regressed.	P0
US-22	As a PM, I want to filter failures by metric, severity, tag, model version, prompt version, and reviewer so that I can identify patterns.	P0

Production Monitoring

ID	User Story	Priority
US-23	As a PM, I want to sample production conversations for eval so that I can monitor real-world quality.	P1
US-24	As an engineer, I want to configure sampling rates so that production eval cost is controlled.	P1

US-25	As a PM, I want failed production examples added to the dataset so that future releases are tested against real failures.	P1

---

11. Recommended Metrics

EvalPilot should support the following metric categories.

Quality Metrics

Metric	Definition
Correctness Score	Measures whether the answer is factually or procedurally correct.
Helpfulness Score	Measures whether the answer solves the user's problem.
Relevance Score	Measures whether the answer addresses the user's intent.
Completeness Score	Measures whether the answer includes all required information.
Tone Score	Measures whether the answer matches the desired brand or support tone.

Trust and Safety Metrics

Metric	Definition
Hallucination Rate	Percentage of responses that include unsupported or fabricated claims.
Groundedness Score	Measures whether the response is supported by retrieved context or source material.
Safety Failure Rate	Percentage of responses that violate safety or policy rules.

Escalation Accuracy	Measures whether the AI escalates when required.
Policy Compliance Score	Measures whether the response follows business policy.

RAG-Specific Metrics

For RAG applications, EvalPilot should support metrics such as faithfulness, response relevancy, context precision, context recall, and retrieval relevance. Ragas documentation lists RAG metrics including context precision, context recall, response relevancy, faithfulness, multimodal faithfulness, and multimodal relevance.

Agent-Specific Metrics

Metric	Definition
Tool Call Accuracy	Measures whether the agent selected the right tool.
Tool Call F1	Measures precision and recall of tool calls.
Task Completion Rate	Measures whether the agent completed the user goal.
Step Efficiency	Measures whether the agent used too many or unnecessary steps.
Recovery Rate	Measures whether the agent recovered after an error.

Ragas also lists agent/tool-use metrics such as topic adherence, tool call accuracy, tool call F1, and agent goal accuracy.

---

12. Hallucination Rate Definition

Hallucination Rate = Number of hallucinated responses / Total evaluated responses × 100

A response should be marked as hallucinated when it includes information that is:

1. Not supported by retrieved context.
2. Not present in the source of truth.
3. Fabricated or unverifiable.
4. Contradictory to known business policy.
5. Overconfident despite missing evidence.

Example

If 500 AI responses are evaluated and 35 include unsupported claims:

Hallucination Rate = 35 / 500 × 100 = 7%

MVP Hallucination Detection Methods

1. Human reviewer label.
2. LLM-as-judge groundedness check.
3. Source citation validation.
4. Retrieval context comparison.
5. Rule-based detection for unsupported policy claims.

---

13. Launch Readiness Criteria

An AI feature should be considered ready for launch only if it meets predefined thresholds.

Example launch gate:

Metric	Required Threshold
Overall Pass Rate	≥ 90%
Hallucination Rate	≤ 3%
Safety Failure Rate	0 critical failures
Policy Compliance	≥ 95%
Human Review Agreement	≥ 85%
Regression vs Previous Version	No critical regression

P0/P1 Failure Cases	100% reviewed

---

14. MVP Functional Requirements

FR-01: Project Creation

Users can create an eval project with name, description, AI use case type, owner, reviewers, and quality dimensions.

FR-02: Dataset Management

Users can create, edit, import, export, tag, and version datasets.

FR-03: Golden Test Cases

Users can mark specific examples as goldens. Goldens become the trusted evaluation baseline.

FR-04: Synthetic Test Generation

Users can generate synthetic test cases from existing examples and approve them before use.

FR-05: Rubric Builder

Users can define scoring criteria, scoring scale, severity, pass/fail thresholds, and launch blockers.

FR-06: Eval Execution

Users can run evals against model outputs using API endpoint, uploaded output file, or manual output entry.

FR-07: Automated Evaluators

The platform supports deterministic checks, LLM-as-judge, reference comparison, groundedness checks, and safety checks.

FR-08: Human Review

Users can manually review outputs, assign scores, add comments, and override automated scores.

FR-09: Version Comparison

Users can compare eval results across prompt versions, model versions, retrieval versions, and release candidates.

FR-10: Dashboard

Users can view pass rate, fail rate, metric trends, severity breakdown, top failure reasons, and launch readiness.

FR-11: Failed Case Management

Users can convert failed evals into improvement tickets or export them to Jira, Azure DevOps, GitHub Issues, or CSV.

FR-12: Production Sampling

Users can import or sample production traces and evaluate real-world interactions.

---

15. Non-Functional Requirements

Category	Requirement
Security	Support role-based access control for admin, PM, engineer, reviewer, and viewer.
Privacy	Allow PII redaction before storing production examples.
Auditability	Maintain version history for datasets, rubrics, model versions, and reviewer decisions.
Scalability	Support batch eval runs with thousands of test cases.
Reliability	Eval results should be reproducible with stored model, prompt, rubric, and dataset versions.
Explainability	Every automated score should include a reason or evidence.
Cost Control	Users should configure sampling rate, judge model, max tokens, and batch size.

Integration	Support API, CSV/JSON upload, webhook, and future integrations with CI/CD and observability tools.

---

16. User Experience Flow

Flow 1: Create First Eval

1. User creates project.
2. User selects AI use case type.
3. Platform recommends default metrics.
4. User adds 10–50 golden examples.
5. User defines rubric.
6. User connects model or uploads outputs.
7. User runs eval.
8. User reviews dashboard.
9. User sends failed cases to review queue.

Flow 2: Compare Two Versions

1. User selects dataset.
2. User selects Version A and Version B.
3. Platform runs evals on both versions.
4. Dashboard shows metric deltas.
5. Platform highlights regressions.
6. PM makes go/no-go decision.

Flow 3: Production Feedback Loop

1. System imports sampled production conversations.
2. Evaluators score live outputs.
3. Critical failures trigger alerts.
4. PM reviews failed examples.
5. Failed examples are added to golden dataset.
6. Engineering fixes prompt/model/retrieval.
7. Team reruns offline evals before redeploying.

LangSmith’s evaluation documentation describes a similar feedback-loop idea: production traces can be evaluated online, failing traces can be added to datasets, fixes can be validated offline, and the improved application can be redeployed.

17. Dashboard Requirements

Executive Summary View

- Overall AI Quality Score
- Launch Readiness Status
- Pass/Fail Trend
- Hallucination Rate
- Safety Failure Rate
- Top Failure Categories
- Regression Summary
- Open Critical Issues

PM View

- Quality by user scenario
- Quality by customer intent
- Quality by feature
- Business policy compliance
- Human review notes
- Launch blockers

Engineering View

- Failures by model version
- Failures by prompt version
- Failures by retrieval version
- Judge score distribution
- Latency and cost per eval run
- Reproducibility metadata

Reviewer View

- Pending review queue
- Assigned cases
- Severity filters
- Rubric scoring panel
- Comment history
- Escalation flag

---

18. Success Metrics

Product Adoption Metrics

Metric	Target
Number of active eval projects	10+ pilot projects
Weekly active users	50+ in pilot
Number of golden test cases created	5,000+
Number of eval runs completed	1,000+
Repeat usage rate	60% of teams run evals weekly

Product Quality Metrics

Metric	Target
Time to create first eval	< 30 minutes
Time to review failed cases	50% reduction vs manual process
Regression detection rate	90% of known regressions caught
Human/AI judge agreement	≥ 85% for calibrated rubrics
Critical issue escape rate	Reduced by 50% after adoption

Business Metrics

Metric	Target
Pilot-to-paid conversion	30%+
Monthly recurring revenue	Defined after pricing test
Retention	80%+ team retention after 90 days
Expansion	2+ projects per customer account

19. MVP Prioritization

P0: Must Have

1. Project creation
2. Golden dataset manager
3. Rubric builder
4. CSV/JSON import
5. Eval runner
6. LLM-as-judge scoring
7. Deterministic rule checks
8. Human review queue
9. Results dashboard
10. Version comparison

P1: Should Have

1. Synthetic test generation
2. Production trace sampling
3. Reviewer calibration
4. Jira/Azure DevOps/GitHub export
5. PII redaction
6. Alerting for critical failures
7. Cost tracking

P2: Could Have

1. CI/CD integration
2. Prompt optimization suggestions
3. Multi-modal evals
4. Advanced agent trajectory evaluation
5. Compliance evidence export
6. Custom judge model marketplace

---

20. Suggested MVP Architecture

Frontend

- React / Next.js
- Dashboard UI
- Dataset editor

- Rubric builder
- Review queue
- Eval run comparison

Backend

- Python FastAPI or Node.js
- Eval orchestration service
- Rubric scoring service
- Dataset service
- Review workflow service
- Result aggregation service

Storage

- PostgreSQL for projects, users, rubrics, metadata
- Object storage for large datasets and outputs
- Vector DB optional for semantic failure clustering
- Audit log table for dataset/rubric/version changes

AI Services

- LLM-as-judge integration
- Synthetic test generation
- Explanation generation
- Failure clustering
- PII redaction

Integrations

- OpenAI / Azure OpenAI / Anthropic / Google model endpoints
- LangSmith / Phoenix export or import in future
- Jira / Azure DevOps / GitHub Issues
- Slack or Teams alerts

---

21. Data Model: MVP

Project

- Project ID
- Name
- Description

- Use case type
- Owner
- Team members
- Created date
- Status

Dataset

- Dataset ID
- Project ID
- Name
- Version
- Source
- Tags
- Created by

Test Case

- Test Case ID
- Dataset ID
- Input prompt
- Expected output
- Context/source material
- Tags
- Severity
- Golden flag
- Created by

Rubric

- Rubric ID
- Project ID
- Metric name
- Description
- Scoring scale
- Pass threshold
- Severity
- Launch blocker flag

Eval Run

- Eval Run ID
- Project ID

- Dataset version
- Model version
- Prompt version
- Rubric version
- Run status
- Start/end time
- Cost
- Created by

Eval Result

- Result ID
- Eval Run ID
- Test Case ID
- AI output
- Metric scores
- Pass/fail
- Explanation
- Human override
- Reviewer comments

---

22. Risks and Mitigations

Risk	Impact	Mitigation
LLM judge gives inconsistent scores	Low trust in eval results	Add judge calibration, human review sampling, and rubric versioning.
PMs find setup too complex	Low adoption	Provide templates by use case and guided onboarding.
Eval cost becomes high	Budget concerns	Add sampling, caching, model selection, and cost estimates before runs.
Synthetic test data is low quality	Bad eval coverage	Require human approval before adding synthetic cases to official datasets.

Hallucination detection is imperfect	False confidence	Combine groundedness checks, source validation, and human review for high-risk cases.
Too much focus on scores	Wrong decisions	Show examples, explanations, and severity breakdowns, not just aggregate scores.

---

23. Competitive / Market Notes

EvalPilot should not try to compete only as a developer observability tool. The stronger positioning is:

“The AI quality operating system for Product Managers and cross-functional AI teams.”

Existing tools often emphasize engineering workflows such as traces, datasets, online/offline evaluations, code-based evaluators, LLM-as-judge, and production monitoring. LangSmith supports offline and online evaluation flows, while Phoenix supports deterministic and LLM-as-judge evaluations against traces, datasets, and experiment results.
EvalPilot should differentiate by focusing on:

1. PM-friendly quality definition.
2. Launch readiness scorecards.
3. Human review workflows.
4. Business policy and domain review.
5. Clear connection between eval failures and product decisions.
6. Simple onboarding for non-ML stakeholders.

---

24. MVP Release Plan

Phase 1: Prototype

- Create project
- Upload dataset
- Define rubric
- Run eval on uploaded outputs

- Show results dashboard

Phase 2: Private Beta

- Add LLM-as-judge
- Add human review queue
- Add version comparison
- Add synthetic test generation
- Add export to CSV/Jira/Azure DevOps

Phase 3: Public Beta

- Add production sampling
- Add alerts
- Add PII redaction
- Add team roles
- Add templates for chatbot, RAG, summarization, and agent evals

Phase 4: Enterprise Readiness

- SSO
- Audit logs
- Advanced RBAC
- Compliance reporting
- Data retention controls
- Private model support

---

25. Example AI Eval Templates

Customer Support Bot Template

Metrics:

- Correctness
- Helpfulness
- Empathy
- Policy compliance
- Escalation accuracy
- Hallucination risk

RAG Knowledge Assistant Template

Metrics:

- Faithfulness
- Groundedness
- Answer relevancy
- Context precision
- Context recall
- Citation accuracy

AI Agent Template

Metrics:

- Goal completion
- Tool call accuracy
- Step efficiency
- Safety compliance
- Recovery from error
- User satisfaction

Content Generation Template

Metrics:

- Brand tone
- Factual accuracy
- Completeness
- Originality
- Safety
- Format compliance

---

26. Open Questions

1. Should the first version target enterprise AI teams or individual AI builders?
2. Should the product start as a SaaS web app or developer SDK?
3. Which model providers should be supported first?
4. Should the initial wedge be chatbot evals, RAG evals, or AI agent evals?

5. Should production monitoring be included in MVP or delayed to V2?
6. Should human review be internal-only or support external reviewers?
7. What level of compliance support is required for enterprise customers?

---

27. Recommended MVP Positioning

EvalPilot helps AI product teams move from "the demo looks good" to "we know this AI feature is ready to ship."

It gives Product Managers and engineering teams a shared system to define quality, test AI outputs, detect hallucinations, compare versions, and continuously improve AI products.

---

28. One-Line Pitch

EvalPilot is an AI EvalOps platform that helps product and engineering teams measure, review, and improve the quality of LLM applications before and after launch.

---

29. Elevator Pitch

AI products are hard to test because outputs are variable, subjective, and sometimes unpredictable. EvalPilot gives teams a structured way to define what good looks like, create golden datasets, run automated and human evaluations, detect hallucinations, compare prompt and model versions, and monitor production quality. It turns AI quality from guesswork into a measurable product workflow.

For your first MVP, I recommend starting with **RAG/chatbot evals** because the value is easier to demonstrate: hallucination rate, groundedness, answer relevancy, and source accuracy.